

EtherCAT Protocol Erweiterung

App-basierte HMI als Ersatz für HMI-Controller

Mobile und Desktop-Anwendungsschnittstelle
in EtherCAT-Netzwerken

Stephan Epp

8. März 2026

Inhaltsverzeichnis

1	Einleitung	3
1.1	Problemstellung	3
1.2	Ziel dieser Arbeit	4
1.3	Kostenvergleich	4
2	Stand der Technik	5
2.1	EtherCAT-Protokollübersicht	5
2.2	Standardtopologie und HMI-Anbindung	6
2.3	Consumer-Hardware-Fähigkeiten	6
2.4	Bestehende Ansätze und deren Grenzen	7
3	EHAE-Architektur	7
3.1	Systemübersicht	7
3.2	Interne Struktur des HMI Application Gateway	8
4	Dual-Role-App-Modell	10
4.1	Engineer App: Topologie-Ansicht	10
4.2	Engineer App: Oszilloskop-Ansicht	11
5	Protokoll-Erweiterungsspezifikation	11
5.1	HAG Communication Protocol (HCP)	11
5.1.1	Nachrichtenformat	12
5.2	Variable Mapping Konfiguration (HVM)	13
6	Sicherheitsarchitektur	13
6.1	Authentifizierungsablauf	13
6.2	Token-Management	14
6.3	Rollenbasierte Zugriffskontrolle	15
6.4	Netzwerksicherheit	15

7	Netzwerkintegration	15
7.1	Firewall-Regelwerk	15
7.2	Latenzbudget	16
8	Migrationsstrategie	17
8.0.1	Risikominderung	17
9	Mobile App: Android-Implementierung mit Capacitor	18
9.1	Das Capacitor-Prinzip	18
9.2	Projektstruktur	18
9.3	Capacitor-Konfiguration	19
9.4	Abhängigkeiten und Framework-Version	20
9.5	Build-Workflow	21
9.6	Vorteile der web-getriebenen Mobile-App-Entwicklung	21
9.7	Android-spezifische Anpassungen	22
10	Evaluation	23
10.1	Kumulative Kostenanalyse	23
10.2	Nicht-monetäre Vorteile	23
11	Schlussfolgerung	24
11.1	Ausblick	24

1 Einleitung

Diese Arbeit schlägt eine Erweiterung des EtherCAT-Standards vor, die formal eine neue Klasse von Human-Machine Interfaces (HMI) definiert, die ausschließlich auf mobilen und Desktop-Applikationen basiert. Traditionelle, band-integrierte HMI-Controller sind teuer, wartungsintensiv und können die Anzeigequalität moderner Consumer-Hardware bei weitem nicht erreichen.

Die vorgeschlagene Erweiterung führt einen *App-Based HMI Layer* (ABH) im EtherCAT-Ökosystem ein, der es Tablets und Smartphones erlaubt, vollständige HMI-Verantwortung auf Maschinenebene zu übernehmen, während eine erweiterte Desktop-Variante für Inbetriebnahme und Diagnose durch Applikationsingenieure genutzt wird. Diese Arbeit beschreibt im Detail die Systemarchitektur, die notwendigen Protokollerweiterungen, das Sicherheitsmodell mit Authentifizierung und rollenbasierter Zugriffskontrolle, eine vierstufige Migrationsstrategie sowie ein Referenz-Implementierungsframework.

Die quantitative Kosten-Nutzen-Analyse zeigt eine Einsparung von bis zu 81 % über fünf Jahre bei 20 HMI-Stationen. Der Proposal richtet sich an die EtherCAT Technology Group (ETG) mit dem Ziel, EHAE als formale Erweiterung des EtherCAT Application Layer in den ETG.1000-Standard aufzunehmen.

EtherCAT (Ethernet for Control Automation Technology), standardisiert in IEC 61158 und IEC 61784, ist eines der am weitesten verbreiteten Echtzeit-Industrial-Ethernet-Protokolle weltweit. Mit seiner segment-basierten Topologie, hochgeschwindigkeits-Datenverarbeitung und dem verteilten Uhr-Mechanismus (Distributed Clock) ist EtherCAT zur Basis-Infrastruktur moderner Maschinenautomation geworden [1]. Zykluszeiten von weniger als 100 μ s ermöglichen präzise Bewegungssteuerung und synchronisierte Multi-Achs-Applikationen, die mit klassischen Feldbussystemen nicht realisierbar wären [3].

Die Human-Machine Interface (HMI) — ein kritisches Teilsystem jeder Maschine — wurde jedoch historisch vom Standardisierungsumfang von EtherCAT ausgeschlossen. Dies hat zur Entstehung einer heterogenen HMI-Landschaft geführt, in der proprietäre Lösungen verschiedener Hersteller parallel existieren, ohne gemeinsame Schnittstellen oder Interoperabilität zu gewährleisten.

1.1 Problemstellung

Traditionell rüsten Maschinenbauer jede Produktionslinie (sog. *Band*) mit einem dedizierten HMI-Panel aus: einem eingebetteten Touchscreen-System, das eine proprietäre Runtime ausführt und direkt am Maschinengehäuse montiert ist. Obwohl diese Systeme ihren Zweck grundsätzlich erfüllen, bringen sie erhebliche Nachteile mit sich:

- **Hohe Anschaffungskosten:** Industrielle Panel-PCs liegen im Preisbereich von 800 € bis über 5 000 € pro Einheit, abhängig von Schutzklasse, Displaygröße und Rechnerleistung.
- **Geringe Anzeigequalität:** Auflösungen zwischen 800×600 und 1920×1080 bei eingeschränkter Helligkeit (typisch 350–600 nit) sind der Standard.
- **Proprietärer Runtime-Lock-in:** Jeder Hersteller nutzt eine eigene Entwicklungsumgebung und Runtime (z. B. Siemens WinCC, Beckhoff TwinCAT HMI, Weintek EasyBuilder), was Portierbarkeit und Herstellerunabhängigkeit stark einschränkt.

- **Wartungsaufwand:** Ersatzteilbeschaffung und Software-Kompatibilität bei veralteten Panels stellen langfristig erhebliche Herausforderungen dar.
- **Fehlende Konnektivität:** Integrierte Remote-Monitoring- und Cloud-Anbindungen sind nur mit proprietären Zusatzlösungen realisierbar.
- **Langer Update-Zyklus:** Firmware- und Software-Updates erfordern oft manuelle Eingriffe vor Ort und sind nicht zentral ausrollbar.

Gleichzeitig übersteigen Consumer-Tablets die HMI-Anzeigequalität bei einem Bruchteil der Kosten. Ein modernes iPad Pro erreicht Auflösungen von 2752×2064 Pixeln, Bildwiederholraten von 120 Hz sowie eine Helligkeit von bis zu 1600 nit — Werte, die industrielle Panels nicht annähernd bieten.

1.2 Ziel dieser Arbeit

Diese Arbeit schlägt die **EtherCAT-HMI-App Extension (EHAE)** vor und definiert, wie app-basierte HMI-Clients mit EtherCAT-Mastern integriert werden können, um bandintegrierte HMI-Controller vollständig zu ersetzen. Die zentralen Beiträge dieser Arbeit sind:

1. Definition eines neuen Software-Layers (*HMI Application Gateway, HAG*) im EtherCAT-Master-Stack.
2. Spezifikation eines standardisierten Kommunikationsprotokolls (*HAG Communication Protocol, HCP*) auf Basis von WebSockets.
3. Formalisierung eines Dual-Role-App-Modells mit klar definierten Berechtigungsstufen.
4. Beschreibung eines auf OAuth 2.0, TLS 1.3 und RBAC aufbauenden Sicherheitsmodells.
5. Quantitativer Nachweis der Kosteneinsparungen gegenüber traditionellen Ansätzen.

1.3 Kostenvergleich

Abbildung 1 illustriert den 5-Jahres-Gesamtbetriebskostenvergleich (Total Cost of Ownership, TCO) zwischen einem traditionellen HMI-Panel und einer app-basierten Lösung.

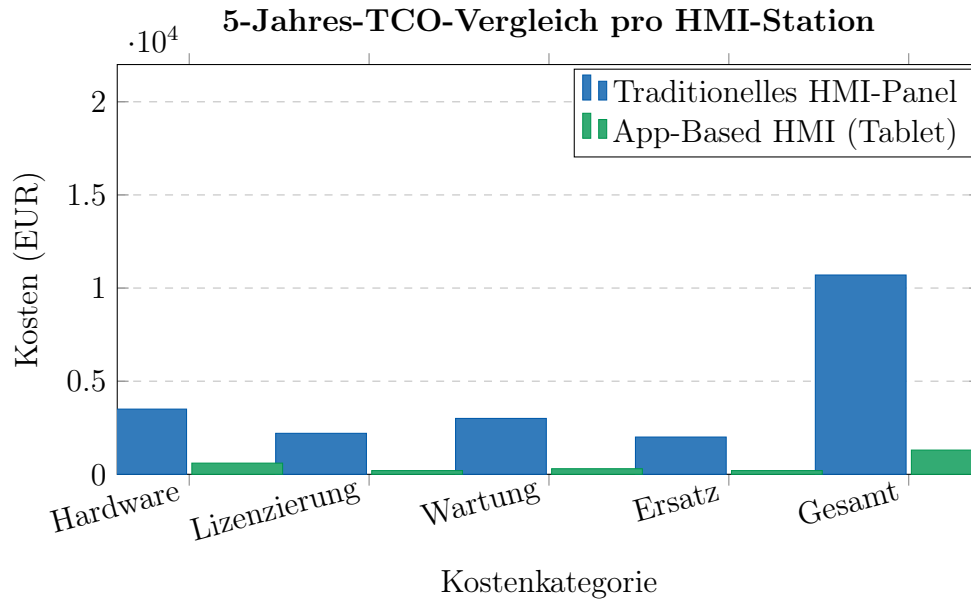


Abbildung 1: 5-Jahres-TCO-Vergleich: traditionelles HMI-Panel vs. app-basiertes HMI pro Station. Das Balkendiagramm zeigt fünf Kostenkategorien (Hardware, Lizenzierung, Wartung, Ersatz und Gesamtkosten) jeweils nebeneinander für beide Varianten. Beim traditionellen HMI-Panel dominieren die Hardware-Kosten mit 3 500 € sowie Wartungskosten von 3 000 €. Die app-basierte Lösung reduziert alle Einzelpositionen drastisch: Hardware auf 600 €, Wartung auf 300 €. In der Gesamtbetrachtung über fünf Jahre ergibt sich 10 700 € gegenüber nur 1 300 € — eine Einsparung von **87 %** pro Station.

Die Differenz zwischen den Varianten ist auf mehrere Faktoren zurückzuführen: Industrielle Panels erfordern Wartungsverträge, herstellerspezifische Schulungen und kostspielige Ersatzteile. App-basierte Systeme auf Consumer-Hardware profitieren von der Skalierung des Massenmarktes, standardisierten App-Store-Distributionen sowie kostenlosen oder preiswerten Entwicklungsframeworks wie Flutter oder .NET MAUI.

2 Stand der Technik

2.1 EtherCAT-Protokollübersicht

EtherCAT operiert auf einem Master-Slave-Prinzip [3]. Ein einzelner Master kommuniziert mit mehreren Slave-Geräten in einer Ring- oder Linientopologie über Standard-Ethernet-Frames und erreicht dabei Zykluszeiten von weniger als 100 μ s. Das Protokoll verwendet einen innovativen *Processing-on-the-fly*-Ansatz: Jeder Slave liest die für ihn relevanten Daten aus dem durchlaufenden Frame heraus und schreibt seine Prozessdaten hinein, während der Frame den Knoten passiert — ohne Pufferung und ohne zusätzliche Latenz.

Die EtherCAT-Protokollarchitektur besteht aus drei wesentlichen Schichten:

1. **Physical Layer (IEC 61158-2):** Definiert die elektrischen und mechanischen Eigenschaften der Übertragung, typischerweise 100BASE-TX über Cat-5-Kabel.
2. **Data Link Layer:** Beinhaltet das Field Memory Management Unit (FMMU) und den SyncManager. Das FMMU ermöglicht das flexible Mapping von Prozessdaten in

den EtherCAT-Frame, während der SyncManager die synchronisierte Kommunikation zwischen Master und Slave verwaltet.

3. **Application Layer:** Definiert höhere Protokolle wie CAN over EtherCAT (CoE), File over EtherCAT (FoE), Ethernet over EtherCAT (EoE) und ADS over EtherCAT (AoE) [2].

2.2 Standardtopologie und HMI-Anbindung

Abbildung 2 zeigt die typische EtherCAT-Netzwerktopologie mit traditionellem HMI-Panel.

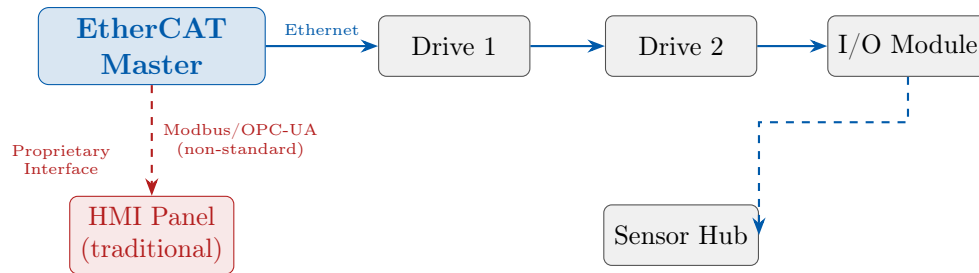


Abbildung 2: Standard-EtherCAT-Topologie mit traditionell angebundenem HMI-Panel. Das Diagramm zeigt den EtherCAT-Master (blau, links oben) als zentralen Kommunikationsknoten, der über eine lineare Ethernet-Kette mit drei Slave-Geräten verbunden ist: Drive 1, Drive 2 und einem I/O-Modul (alle grau). Ein vierter Slave, der Sensor Hub, ist mit gestrichelter Linie als Abzweigung von Drive 2 dargestellt. Das traditionelle HMI-Panel (rot markiert, unten links) ist *nicht* Teil des EtherCAT-Rings, sondern über eine gestrichelte, rot markierte Verbindung mit nicht-standardisierten Protokollen (Modbus oder OPC-UA) am Master angebunden. Dies illustriert das Kernproblem: HMI ist kein nativer Bestandteil des EtherCAT-Standards, sondern eine proprietäre Ergänzung.

Diese Entkopplung hat weitreichende Konsequenzen: Da das HMI außerhalb des EtherCAT-Standards liegt, existiert keine einheitliche Schnittstelle. Jeder Hersteller implementiert die Verbindung zwischen Master und HMI individuell, was zu Kompatibilitätsproblemen, proprietären Protokollen und erhöhtem Integrationsaufwand führt.

2.3 Consumer-Hardware-Fähigkeiten

Der rasante Fortschritt im Consumer-Elektronik-Markt hat eine technologische Lücke gegenüber industriellen HMI-Panels entstehen lassen, die sich Jahr für Jahr vergrößert. Tabelle 1 fasst die wichtigsten Parameter zusammen.

Tabelle 1: Displayvergleich: industrielles HMI-Panel vs. Consumer-Tablet (2024).

Parameter	Industrielles HMI-Panel	Consumer-Tablet (iPad Pro / Galaxy Tab S9)
Auflösung	1024×768 – 1920×1080	2560×1600 – 2752×2064
Bildwiederholrate	60 Hz	60–120 Hz (ProMotion)
Helligkeit	350–600 nit	600–1600 nit
Touch-Präzision	±2–5 mm	±0,5 mm
Preis (EUR)	800–5 000	350–1 300
Prozessor	ARM Cortex-A, 1–2 GHz	Apple M4 / Snapdragon 8 Gen 2
RAM	512 MB – 2 GB	8–16 GB
Akkulaufzeit	entfällt (Netzbetrieb)	10–12 Stunden

Neben den Displayeigenschaften sind Consumer-Tablets in weiteren Dimensionen deutlich überlegen: ihre Prozessoren ermöglichen flüssige Echtzeitvisualisierungen, Trend-Displays und sogar lokale Datenanalyse. Die Integration von Wi-Fi 6, Bluetooth und Mobilfunk bietet Konnektivitätsoptionen, die industrielle Panels nicht besitzen.

2.4 Bestehende Ansätze und deren Grenzen

Bereits heute gibt es Versuche, Consumer-Geräte als HMI einzusetzen. Diese basieren typischerweise auf OPC-UA-Konnektivität oder proprietären Herstellerlösungen wie Siemens MindSphere oder Beckhoff TF2000. Sie weisen jedoch folgende Schwächen auf:

- Keine formale Standardisierung im EtherCAT-Kontext.
- Fehlende Definition von Sicherheitsrollen und Zugriffsrechten.
- Keine spezifizierte Latenzanforderung für die HMI-App-Konnektivität.
- Vendor-Lock-in durch proprietäre Protokolle.

EHAe schließt diese Lücke durch eine vollständige, herstellerunabhängige Spezifikation.

3 EHAe-Architektur

3.1 Systemübersicht

Die EHAe definiert einen neuen Software-Layer — den *HMI Application Gateway (HAG)* — innerhalb des EtherCAT-Master-Stacks. Der HAG exponiert eine abgesicherte, WebSocket-basierte API, mit der HMI-App-Clients über Wi-Fi oder kabelgebundenes Ethernet verbinden können.

Abbildung 3 zeigt das vollständige Schichtenmodell der EHAe.

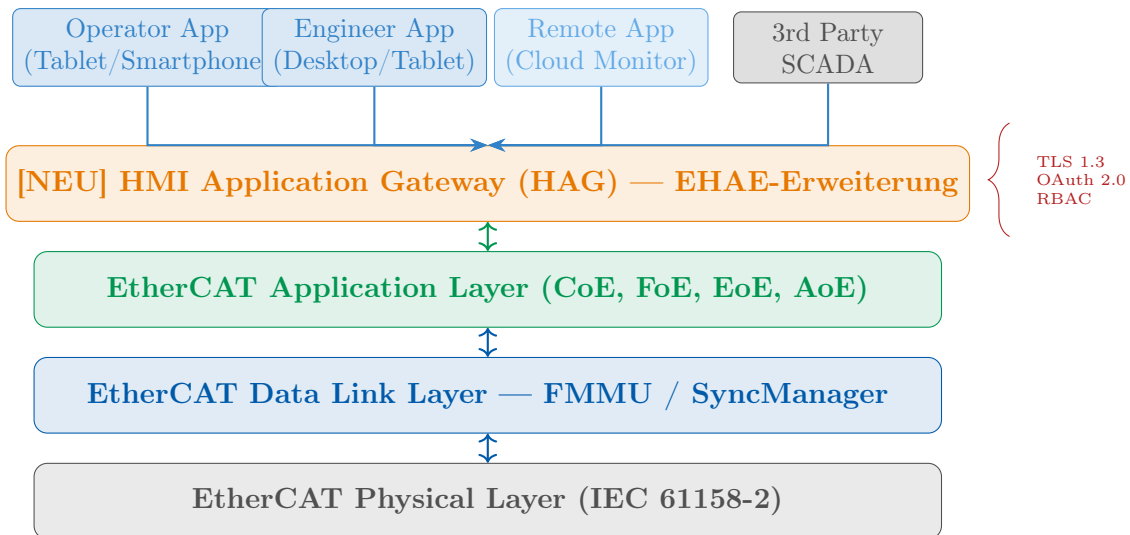


Abbildung 3: EHAE-Architekturdiagramm: Schichtenmodell der erweiterten EtherCAT-Protokollstack. Das Diagramm ist von unten nach oben aufgebaut und zeigt vier aufeinander aufbauende Schichten. Ganz unten (grau) liegt der *EtherCAT Physical Layer* gemäß IEC 61158-2. Darüber (blau) befindet sich der *Data Link Layer* mit FMMU und SyncManager. Die dritte Schicht (grün) ist der *Application Layer* mit CoE, FoE, EoE und AoE. Die vierte, orange hervorgehobene Schicht ist die neue EHAE-Erweiterung: der *HMI Application Gateway (HAG)*. Von dieser Schicht aus verbinden sich vier Typen von App-Clients über WebSocket-Verbindungen (Pfeile von oben): die Operator App (Tablet/Smartphone), die Engineer App (Desktop/Tablet), die Remote App (Cloud-Monitor) sowie Drittanbieter-SCADA-Systeme. Rechts neben dem HAG zeigt eine geschweifte Klammer die eingesetzten Sicherheitsmechanismen: TLS 1.3, OAuth 2.0 und RBAC.

Die zentrale Design-Entscheidung des EHAE-Architekturmodells besteht darin, den HAG *oberhalb* des bestehenden Application Layers zu positionieren. Dies hat mehrere Vorteile:

- **Keine Änderungen an Slave-Firmware:** Alle Erweiterungen laufen ausschließlich im Master-Stack.
- **Rückwärtskompatibilität:** Bestehende EtherCAT-Netzwerke können ohne Modifikation der Feldgeräte auf EHAE migrieren.
- **Herstellerfreiheit:** Jeder EtherCAT-Master-Stack-Anbieter kann den HAG unabhängig implementieren.

3.2 Interne Struktur des HMI Application Gateway

Abbildung 4 zeigt die interne Modulstruktur des HAG.

Jedes Modul des HAG hat eine klar definierte Verantwortlichkeit:

Config Loader Liest und parst die HMI Variable Map (HVM, siehe Abschnitt 5) und konfiguriert daraus den PDO Mapper zur Laufzeit.

PDO Mapper Führt das zyklische Mapping von EtherCAT-Prozessdatenobjekten (PDOs) auf benannte HMI-Variablen durch. Erkennt Werteänderungen und generiert entsprechende Events.

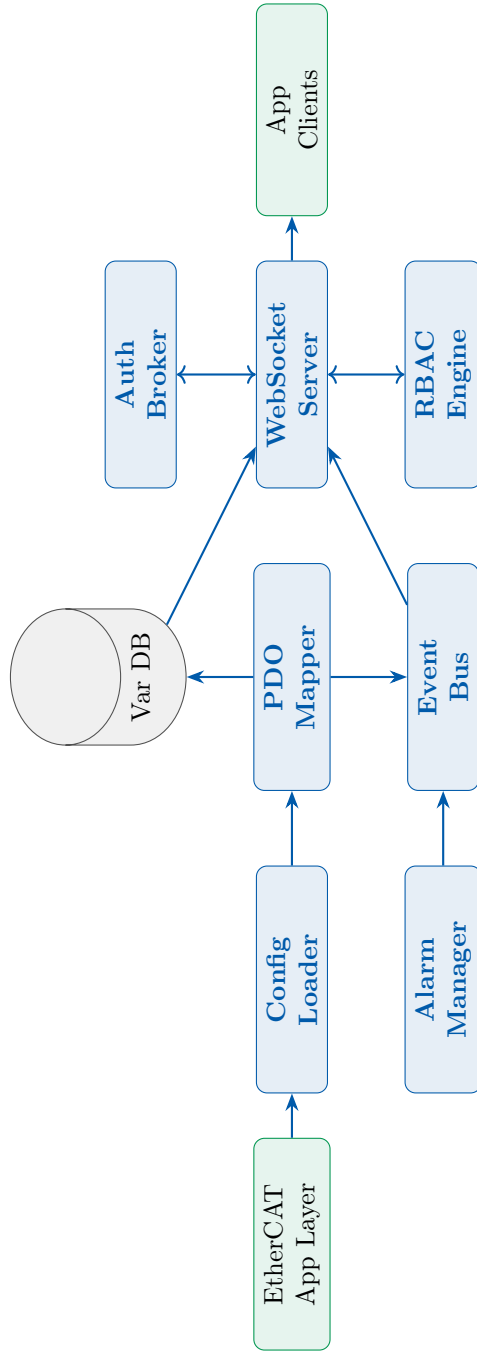


Abbildung 4: Interne Modulstruktur des HAG. Das Blockdiagramm zeigt sieben funktionale Module (blau) sowie eine zentrale Datenbank (grau, Zylinder-Symbol) und zwei externe Schnittstellen (grün). Von links kommend empfängt der *Config Loader* die Prozessdaten-Mappings aus dem EtherCAT Application Layer. Der *PDO Mapper* übersetzt rohe EtherCAT-Prozessdaten in benannte HMI-Variablen und speichert diese in der *Variable Database (Var DB)*. Der *Websocket Server* in der Mitte ist das Verbindungsstück zu den App-Clients (rechts). Er kommuniziert bidirektional mit dem *Auth Broker* (Token-Validierung) und der *RBAC Engine* (Zugriffsrechtsprüfung). Der *Event Bus* sammelt Ereignisse aus dem PDO Mapper und dem Alarm Manager und leitet sie an den Websocket Server weiter, der sie als Push-Nachrichten an die Clients ausliefert.

Variable Database In-Memory-Datenbank, die den aktuellen Zustand aller HMI-Variablen hält. Dient als konsistenter Snapshot-Speicher für neue WebSocket-Verbindungen.

WebSocket Server Verwaltet alle aktiven Client-Verbindungen, verarbeitet eingehende Nachrichten und verteilt ausgehende Updates asynchron.

Auth Broker Validiert JWT-Tokens bei Verbindungsaufbau via Token-Introspection-Endpoint des Identity Providers.

RBAC Engine Prüft vor jeder schreibenden Operation, ob die Rolle des Clients die erforderliche Berechtigung besitzt.

Event Bus Internes Publish-Subscribe-System für lose Kopplung zwischen Modulen.

Alarm Manager Verwaltet den Alarm-Lifecycle (raise, acknowledge, clear) und persistiert die Alarm-Historie.

4 Dual-Role-App-Modell

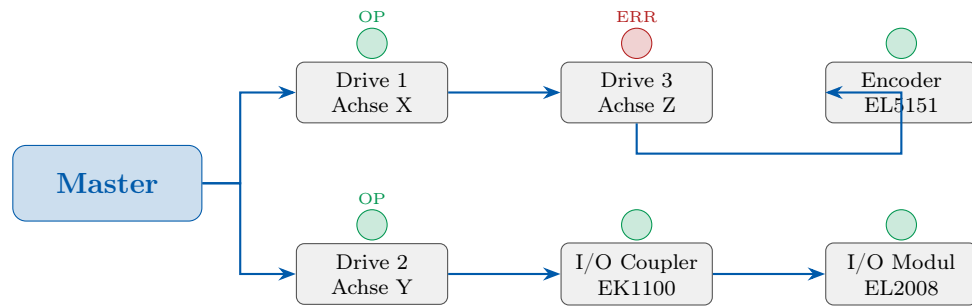
EHAE definiert formal zwei App-Rollen, die auf demselben Anwendungsframework laufen, aber unterschiedliche Capability-Sets besitzen. Dieses Modell erlaubt es, eine einzige Codebase für beide Rollen zu nutzen und die Fähigkeiten dynamisch auf Basis des OAuth-Tokens zur Laufzeit zu aktivieren.



Abbildung 5: Dual-Role-App-Modell: Gegenüberstellung von Operator App und Engineer App. Das Diagramm zeigt drei Geräteklassen von links nach rechts, deren Capability-Sets streng hierarchisch aufgebaut sind. Links (blau) steht die *Operator App* auf einem Smartphone: sie erlaubt grundlegende Steuerungsoperationen (EIN/AUS), die Statusanzeige und das Quittieren von Alarmen; Parameteränderungen sind explizit gesperrt (✗). In der Mitte (grün) die *Operator App* auf einem Tablet mit erweiterter Anzeigefläche: alle Smartphone-Features plus Trendanzeige und Rezeptauswahl sind verfügbar. Rechts (grau) die *Engineer App* auf Desktop oder Tablet: das vollständige Capability-Set einschließlich SDO-Parametrierung, Antriebsinbetriebnahme, Firmware-Updates über FoE und Echtzeitoszilloskop.

4.1 Engineer App: Topologie-Ansicht

Eine der leistungstärksten Funktionen der Engineer App ist die Live-Topologie-Visualisierung. Sie zeigt das gesamte EtherCAT-Netzwerk in Echtzeit, inklusive Gerätezustände, Diagnoseinformationen und Netzwerkstatistiken.



Netzwerkzyklus: 250 µs | Working Counter: 12/12 | Frame Loss: 0

Abbildung 6: Engineer-App-Topologieansicht: Live-EtherCAT-Netzwerkkarte mit Gerätezuständen. Das Diagramm zeigt den EtherCAT-Master (blau) links, von dem zwei parallele Stränge abzweigen. Der obere Strang führt über Drive 1 (Achse X, grün = Betrieb) zu Drive 3 (Achse Z, **rot** = **Fehler**), der untere über Drive 2 (Achse Y) zu I/O-Coupler EK1100 und I/O-Modul EL2008. Der Encoder EL5151 ist als Abzweigung von Drive 3 dargestellt. Über jedem Slave-Knoten ist ein farbiger Kreis mit Statuskürzel dargestellt: **grün OP** (Operational) für alle funktionierenden Geräte, **rot ERR** für Drive 3 (Achse Z), das einen Fehler meldet. Die Statuszeile am unteren Rand zeigt Netzwerkparameter: Zykluszeit 250 µs, Working Counter 12/12 (alle Slaves antworten), Frame Loss 0.

Der Working Counter (WC) ist ein kritischer Diagnoseparameter in EtherCAT-Netzwerken. Er zählt, wie viele Slaves korrekt auf den umlaufenden Frame geantwortet haben. Ein WC von 12/12 bedeutet, dass alle erwarteten 12 Datagramm-Antworten eingetroffen sind. Abweichungen deuten auf Kommunikationsfehler oder ausgefallene Slaves hin. Die Engineer App visualisiert diesen Parameter dauerhaft und schlägt Alarm bei Unstimmigkeiten.

4.2 Engineer App: Oszilloskop-Ansicht

Die Engineer App bietet eine vollständige Oszilloskop-Funktion, die Prozessvariablen in Echtzeit mit einer konfigurierbaren Abtastrate aufzeichnet. Typische Anwendungsfälle sind:

- Analyse von Regelkreisverhalten (Überschwingen, Einregelzeit, Stationäre Abweichung).
- Diagnose von mechanischen Resonanzen durch FFT-Analyse.
- Inbetriebnahme von Lageregelungen (Position, Geschwindigkeit, Strom).
- Aufzeichnung von Anlaufkurven nach Parameteränderungen.

5 Protokoll-Erweiterungsspezifikation

5.1 HAG Communication Protocol (HCP)

Die EHAE definiert das *HAG Communication Protocol (HCP)*, das über WebSocket (RFC 6455) [4] mit JSON- oder MessagePack-Serialisierung arbeitet. Die Wahl von WebSocket über alternatives HTTP/REST ist bewusst getroffen: WebSocket unterstützt

Full-Duplex-Kommunikation und Server-seitige Push-Events, was für eine reaktive HMI mit geringer Latenz unverzichtbar ist.

Tabelle 2 listet alle definierten HCP-Nachrichtentypen auf.

Tabelle 2: HCP-Nachrichtentypen.

Nachricht	Richtung	Beschreibung
SUBSCRIBE	Client → HAG	Abonnement benannter HMI-Variablen
DATA_UPDATE	HAG → Client	Push aktualisierter Variablenwerte
WRITE_VAR	Client → HAG	Variable schreiben (rollengesperrt)
ALARM_EVENT	HAG → Client	Alarm ausgelöst oder quittiert
ALARM_ACK	Client → HAG	Operator quittiert Alarm
SDO_READ/WRITE	Client → HAG	SDO-Zugriff (nur Engineer-Rolle)
TOPO_REQUEST	Client → HAG	Topologie-Snapshot anfordern
HEARTBEAT	Bidirektional	Keep-Alive mit Zeitstempel
SCOPE_SUBSCRIBE	Client → HAG	Oszilloskop-Stream starten
SCOPE_DATA	HAG → Client	Hochfrequente Messdaten (bis 1 kHz)
FOE_UPLOAD	Client → HAG	Firmware-Upload initiieren (FoE)
FOE_STATUS	HAG → Client	Upload-Fortschritt und Status

5.1.1 Nachrichtenformat

Jede HCP-Nachricht folgt einem einheitlichen JSON-Schema:

Listing 1: HCP-Nachrichtenformat (Beispiel: DATA_UPDATE).

```

1 {
2   "type": "DATA_UPDATE",
3   "seq": 1042,
4   "ts": 1712345678.342,
5   "variables": [
6     {
7       "name": "axis_x.actual_velocity",
8       "value": 1247.5,
9       "unit": "rpm",
10      "quality": "GOOD"
11    },
12    {
13      "name": "axis_x.motor_temp",
14      "value": 72.1,
15      "unit": "degC",
16      "quality": "GOOD"
17    }
18  ]
19 }
```

Das Feld quality folgt der OPC-UA-Qualitätssemantik (GOOD, UNCERTAIN, BAD) und erlaubt der App, fehlerhafte oder veraltete Werte visuell zu kennzeichnen.

5.2 Variable Mapping Konfiguration (HVM)

Die HMI Variable Map (HVM) ist eine XML-Konfigurationsdatei, die die Abbildung von EtherCAT-PDO/SDO-Indizes auf benannte HMI-Variablen beschreibt. Sie definiert Datentyp, Skalierung, Einheit, Zugriffsrechte und die erforderliche Mindestrolle für jeden Parameter.

Listing 2: HMI Variable Map (HVM) Konfigurationsbeispiel.

```
1 <HMIVariableMap xmlns="urn:ethercat:ehae:hvm:1.0">
2   <Device slaveAddress="1001" name="Drive_Axis_X">
3     <Variable
4       hmiName="axis_x.actual_velocity"
5       pdoIndex="0x6044" pdoSubIndex="0x00"
6       dataType="INT16" scale="0.1" unit="rpm"
7       access="READ" minRole="OPERATOR" />
8     <Variable
9       hmiName="axis_x.target_velocity"
10      pdoIndex="0x60FF" pdoSubIndex="0x00"
11      dataType="INT32" unit="rpm"
12      access="READWRITE" minRole="OPERATOR" />
13    <Variable
14      hmiName="axis_x.kp_gain"
15      sdoIndex="0x60F6" sdoSubIndex="0x01"
16      dataType="UINT16"
17      access="READWRITE" minRole="ENGINEER" />
18    <Variable
19      hmiName="axis_x.motor_temp"
20      pdoIndex="0x2010" pdoSubIndex="0x03"
21      dataType="INT16" scale="0.1" unit="C"
22      access="READ" minRole="OPERATOR"
23      alarmHigh="80.0" warnHigh="75.0" />
24   </Device>
25 </HMIVariableMap>
```

Die Erweiterung um alarmHigh/warnHigh-Attribute erlaubt es dem Alarm Manager, Grenzwertüberschreitungen direkt aus der HVM-Konfiguration abzuleiten, ohne dass diese Logik in der App selbst implementiert werden muss.

6 Sicherheitsarchitektur

Die Sicherheitsarchitektur der EHAE basiert auf drei aufeinander aufbauenden Säulen: Transport-Verschlüsselung (TLS 1.3), Authentifizierung (OAuth 2.0 mit PKCE) und Autorisierung (Role-Based Access Control, RBAC).

6.1 Authentifizierungsablauf

Abbildung 7 zeigt die vollständige OAuth-2.0-PKCE-Sequenz für den Verbindungsaufbau einer HMI-App zum HAG.

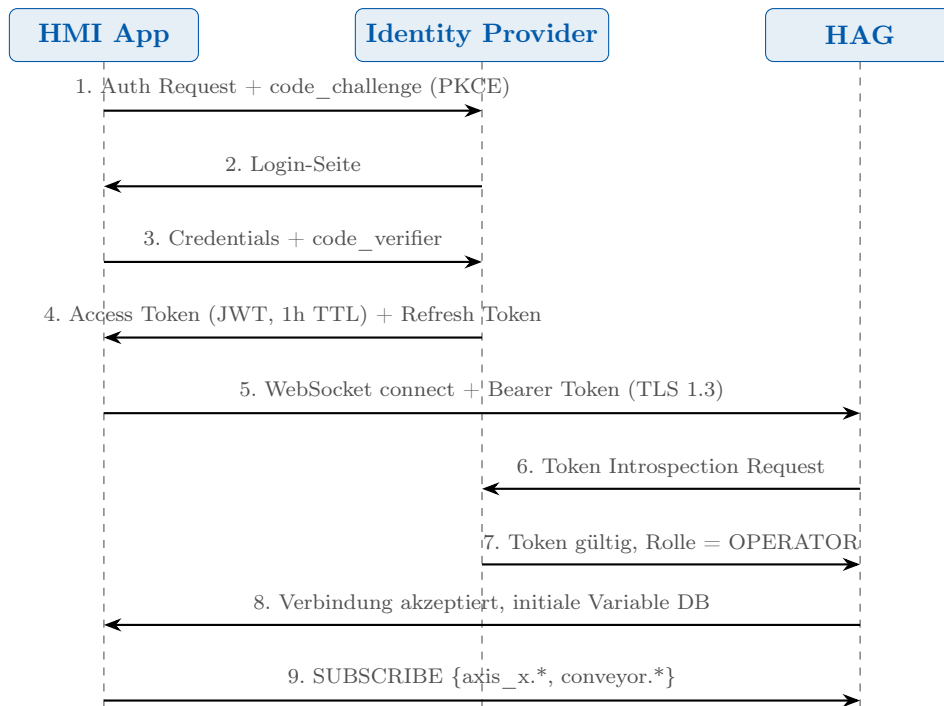


Abbildung 7: EHAE-Authentifizierungssequenz: OAuth 2.0 mit PKCE (Proof Key for Code Exchange, RFC 7636). Das Sequenzdiagramm zeigt drei Akteure auf vertikalen gestrichelten Lebenslinien: die *HMI App* (links), den *Identity Provider* (Mitte) und den *HAG* (rechts). In Schritt 1 sendet die App einen Auth-Request mit einem PKCE-Code-Challenge an den Identity Provider. Dieser antwortet mit einer Login-Seite (Schritt 2). Nach Eingabe der Credentials und Übermittlung des Code-Verifiers (Schritt 3) stellt der Identity Provider ein JWT-Access-Token mit 1 Stunde Laufzeit und ein Refresh-Token aus (Schritt 4). In Schritt 5 verbindet sich die App über TLS 1.3 mit dem HAG und übergibt das Bearer-Token. Der HAG verifiziert das Token durch Token-Introspection beim Identity Provider (Schritte 6–7) und akzeptiert die Verbindung mit Übertragung der initialen Variable-Datenbank (Schritt 8). Abschließend abonniert die App ihre benötigten Variablen (Schritt 9).

Die Verwendung von PKCE [5] ist entscheidend für die Sicherheit auf mobilen Geräten, da diese keine Client-Secrets sicher speichern können. PKCE stellt sicher, dass nur der Initiator des Auth-Flows das Token einlösen kann, selbst wenn der Authorization Code abgefangen wird.

6.2 Token-Management

JWTs enthalten folgende standardisierten Claims gemäß RFC 7519:

- **sub**: Eindeutige Benutzer-ID.
- **iat/exp**: Ausstellungs- und Ablaufzeit (1 Stunde TTL).
- **ehae_role**: EHAE-spezifischer Claim für die Rolle (VIEWER, OPERATOR, ENGINEER, ADMIN).
- **ehae_machines**: Whitelist der Maschinen-IDs, auf die dieser Token Zugriff gewährt.

Nach Ablauf des Access Tokens verwendet die App das Refresh Token, um still im Hintergrund ein neues Access Token zu erhalten, ohne den Bediener mit einem erneuten Login zu unterbrechen.

6.3 Rollenbasierte Zugriffskontrolle

Tabelle 3 zeigt die vollständige RBAC-Matrix für alle vier EHAE-Rollen.

Tabelle 3: EHAE-RBAC-Matrix.

Fähigkeit	Viewer	Operator	Engineer	Admin
Prozessdaten lesen	✓	✓	✓	✓
Alarmer quittieren		✓	✓	✓
Sollwerte schreiben (PDO)		✓	✓	✓
Rezepte laden		✓	✓	✓
SDO-Parameter lesen/schreiben			✓	✓
Firmware-Update (FoE)			✓	✓
Oszilloskop-Stream			✓	✓
Benutzerverwaltung				✓
Rollenänderung				✓

Die RBAC-Engine prüft bei jeder eingehenden `WRITE_VAR`- oder `SDO_WRITE`-Nachricht sowohl die Rolle des Clients als auch die `minRole`-Definition der jeweiligen Variable in der HVM-Konfiguration. Eine Schreiboperation wird nur ausgeführt, wenn beide Bedingungen erfüllt sind.

6.4 Netzwerksicherheit

Alle WebSocket-Verbindungen sind obligatorisch über TLS 1.3 [6] gesichert. TLS 1.3 bietet gegenüber TLS 1.2 wichtige Verbesserungen:

- Reduzierter Handshake auf 1 Round-Trip-Time (1-RTT).
- Entfernung unsicherer Cipher Suites (RC4, 3DES, etc.).
- Forward Secrecy für alle Verbindungen durch ephemere Diffie-Hellman-Schlüssel.
- 0-RTT-Wiederverbindung für bekannte Server (mit Replay-Schutz).

7 Netzwerkintegration

Die VLAN-Segmentierung ist eine zentrale Sicherheitsmaßnahme: Sie verhindert, dass kompromittierte HMI-Geräte direkten Zugriff auf das EtherCAT-Produktionsnetz erhalten. Der gesamte Datenfluss läuft ausschließlich über den HAG, der als einziger Kommunikationspunkt zwischen App-Clients und EtherCAT-Netz fungiert.

7.1 Firewall-Regelwerk

Das empfohlene Firewall-Regelwerk erlaubt folgende Verbindungen:

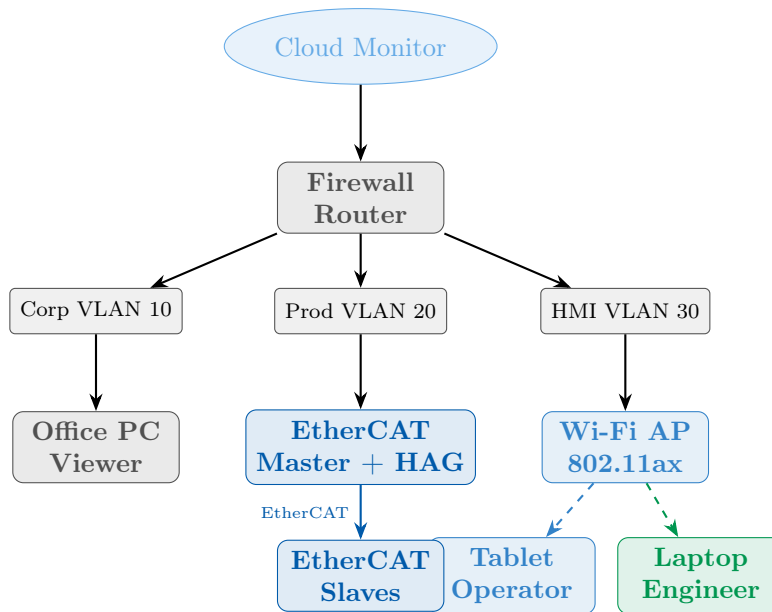


Abbildung 8: Empfohlene Netzwerksegmentierung für EHAe mit drei VLANs. Das Diagramm zeigt die vollständige Netzwerktopologie von oben nach unten. Ganz oben steht der *Cloud Monitor* (blau, Ellipse), der über einen zentralen *Firewall/Router* (grau) mit drei separaten VLANs verbunden ist: VLAN 10 für das Unternehmens-/Büronetz (links), VLAN 20 für das Produktionsnetz (Mitte) und VLAN 30 für das HMI-Netz (rechts). Im Produktionsnetz (VLAN 20) befindet sich der *EtherCAT Master mit HAG* sowie die direkt angebotenen *EtherCAT Slaves*. Im HMI-Netz (VLAN 30) steht ein *Wi-Fi Access Point 802.11ax*, über den *Operator-Tablet* und *Engineer-Laptop* (gestrichelte Pfeile = WLAN) verbunden sind. Im Büronetz (VLAN 10) ist ein *Office PC* für die Viewer-Rolle angebunden. Die Firewall kontrolliert den Zugriff zwischen den VLANs und zur Cloud.

- **VLAN 30 → VLAN 20, Port 8443:** WebSocket-Verbindungen der HMI-Apps zum HAG (TCP, TLS 1.3).
- **VLAN 10 → VLAN 20, Port 8443:** Viewer-Zugriff vom Office-PC.
- **VLAN 20 → Internet, Port 443:** Token-Introspection zum Cloud-IdP.
- **Alle anderen:** Default DENY.

7.2 Latenzbudget

Tabelle 4 zeigt das End-to-End-Latenzbudget für die app-basierte HMI.

Tabelle 4: End-to-End-Latenzbudget für app-basiertes HMI.

Stufe	Beschreibung	Typisch	Maximum
EtherCAT-Zyklus	PDO-Datenaktualisierung	250 μ s	1 ms
HAG-Verarbeitung	Mapping, Änderungserkennung	0,1 ms	0,5 ms
WebSocket-Framing	JSON-Serialisierung	0,5 ms	2 ms
Wi-Fi 6	Drahtlose Übertragung	1–3 ms	10 ms
App-Rendering	UI-Aktualisierung	8–16 ms	33 ms
Gesamt E2E	Prozessdaten bis Display	$\approx$ 10 ms	$\approx$ 47 ms

Eine Gesamtlatenz von typisch 10 ms ist für alle HMI-Anwendungsszenarien vollkommen ausreichend. Die menschliche Wahrnehmungsschwelle für visuelle Reaktionen liegt bei ca. 100 ms; damit bietet EHAE einen komfortablen 10-fachen Sicherheitsabstand. Lediglich für hochdynamische Steuerungseingriffe (Not-Halt etc.) muss beachtet werden, dass Not-Halt-Funktionen weiterhin direkt am Feldbus oder als Safety-Funktion (STO) im Antrieb implementiert werden müssen — nicht über die HMI-App.

8 Migrationsstrategie

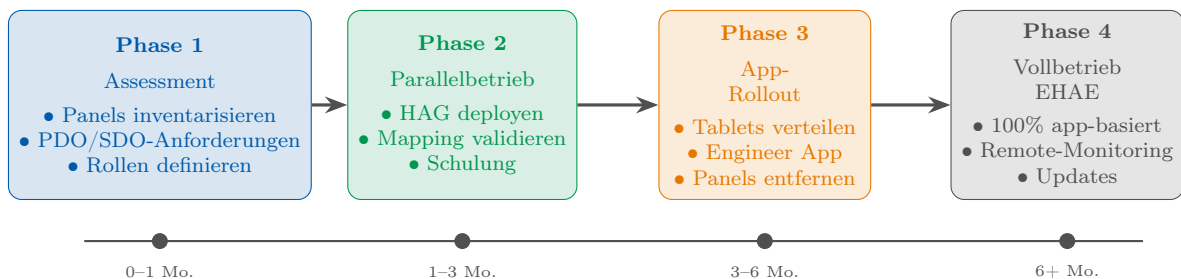


Abbildung 9: Vierstufige Migrationsstrategie von traditionellen HMI-Panels zu EHAE. Das Phasendiagramm zeigt vier sequentielle Phasen auf einer Zeitachse. **Phase 1 (blau, 0–1 Monat):** Assessment — alle vorhandenen HMI-Panels werden inventarisiert, PDO/SDO-Anforderungen analysiert und Benutzerrollen definiert. **Phase 2 (grün, 1–3 Monate):** Parallelbetrieb — der HAG wird deployt und parallel zum bestehenden HMI-Panel betrieben; das Variable-Mapping wird validiert und Mitarbeiter werden geschult. **Phase 3 (orange, 3–6 Monate):** App-Rollout — Tablets werden an Operatoren verteilt, die Engineer App wird für alle Inbetriebnahmen genutzt, und die traditionellen Panels werden schrittweise entfernt. **Phase 4 (grau, ab 6 Monate):** Vollbetrieb EHAE — 100 % app-basierter Betrieb mit Remote-Monitoring und Over-the-Air-Updates.

Die Migrationsstrategie ist so konzipiert, dass zu keinem Zeitpunkt ein Produktionsausfall entsteht. In Phase 2 laufen beide Systeme parallel, und erst nach vollständiger Validierung des app-basierten HMI werden die Panels abgebaut. Dies minimiert das Risiko und erlaubt einen kontrollierten Übergang auch in laufenden Produktionsumgebungen.

8.0.1 Risikominderung

Folgende Maßnahmen reduzieren das Migrationsrisiko:

- **Fallback-Plan:** Während Phase 2 und 3 bleibt das traditionelle Panel als Rückfall-option aktiv.
- **Pilotmaschine:** Die Migration beginnt an einer unkritischen Pilotmaschine, bevor sie auf die gesamte Produktionslinie ausgerollt wird.
- **Schrittweise Rollenfreigabe:** Operators erhalten zunächst nur Lesezugriff; Schreibrechte werden nach Schulungsabschluss freigeschaltet.
- **Audit-Log:** Alle Schreiboperationen und Alarmereignisse werden im HAG protokolliert und sind für Audits nachverfolgbar.

9 Mobile App: Android-Implementierung mit Capacitor

Der EHAE-Referenz-Implementierung liegt eine entscheidende Architekturentscheidung zugrunde: Die Mobile App für Android ist *keine* eigenständige Entwicklung, sondern eine direkte Kapselung der bereits vollständig funktionsfähigen Web-App in eine native Android-Anwendung — realisiert durch das **Capacitor-Framework** [7] von Ionic. Dieses Prinzip der *web-getriebenen Mobile-App-Entwicklung* maximiert den Code-Reuse und minimiert den Wartungsaufwand erheblich: jede Änderung an der Web-App steht automatisch auch in der Mobile App zur Verfügung, ohne dass eine zweite Codebase gepflegt werden muss.

9.1 Das Capacitor-Prinzip

Capacitor ist ein plattformübergreifendes App-Laufzeit-Framework, das Web-Apps (HTML, CSS, JavaScript) als vollwertige native Apps auf Android und iOS ausführt. Im Gegensatz zu einem klassischen Hybrid-App-Ansatz (z. B. Cordova/PhoneGap) bettet Capacitor die Web-App in eine native **WebView**-Komponente (`android.webkit.WebView`) ein und stellt eine JavaScript-Bridge zur Verfügung, über die Web-Code auf native Geräteschnittstellen zugreifen kann.

Die entscheidende Konsequenz dieses Modells: Die gesamte EHAE-Applikationslogik — Dashboard, Topologie-Viewer, Alarm-Manager, Oszilloskop, SDO-Browser, TCO-Kostenanalyse — ist *ausschließlich* in der Web-App implementiert und muss nur *einmal* entwickelt und gewartet werden. Capacitor fungiert dabei als dünner Wrapper, der die Web-App in eine vollwertige Android-APK verwandelt.

9.2 Projektstruktur

Die Android-App-Implementierung liegt unter `src/android/ehae-android/` und folgt der Standard-Capacitor-Projektstruktur:

Listing 3: Projektstruktur der EHAE-Android-App.

```

1 ehae-android/
2 |-- www/                # Web-Assets (EHAE HMI SPA)
3 |   '-- index.html      # Vollstaendige EHAE Web-App
4 |-- android/            # Natives Android-Gradle-Projekt
5 |   |-- app/
6 |   |   |-- src/main/    # Android Activity, XML-Manifest
7 |   |   '-- build.gradle # App-Level Build-Konfiguration

```

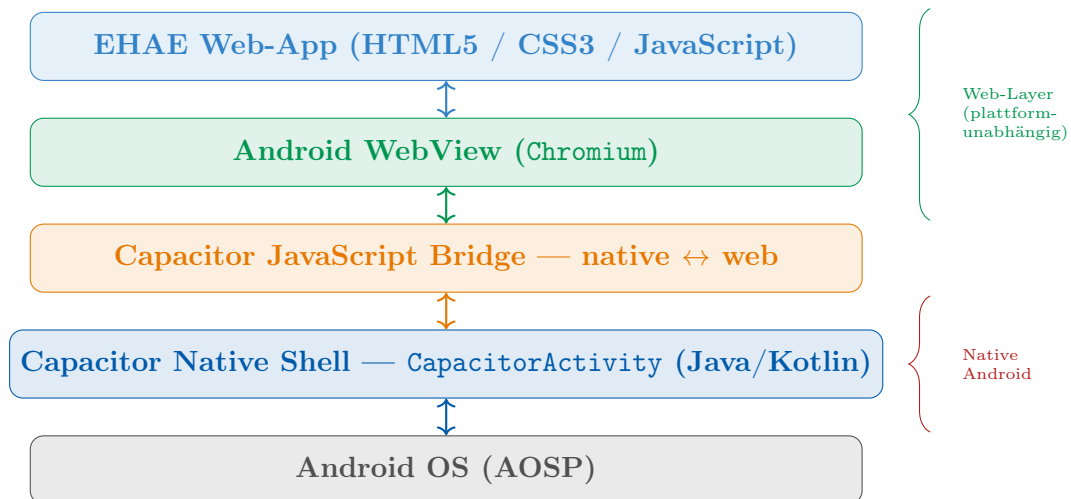


Abbildung 10: Capacitor-Schichtenmodell der EHAE-Android-App. Das Diagramm zeigt fünf aufeinander aufbauende Schichten von unten nach oben. Ganz unten (grau) das *Android-Betriebssystem (AOSP)*. Darüber (blau) die *Capacitor Native Shell*: eine *CapacitorActivity*, die den Android-Lebenszyklus verwaltet. Die dritte Schicht (orange) ist die *Capacitor JavaScript Bridge*: sie ermöglicht den bidirektionalen Datenaustausch zwischen nativem Kotlin/Java-Code und dem JavaScript der Web-App. Die vierte Schicht (grün) ist die *Android WebView* auf Chromium-Basis, die die Web-App rendert. Die oberste Schicht (blau) ist die eigentliche *EHAE Web-App* — dieselbe *index.html*, die auch als Browser-Anwendung genutzt wird. Rechts zeigen zwei geschweifte Klammern die Trennung zwischen nativem Android-Layer und dem plattformunabhängigen Web-Layer.

```

8 | | -- capacitor-cordova-android-plugins/
9 | | -- build.gradle           # Projekt-Level Build-Konfiguration
10 | -- capacitor.config.json   # Capacitor-Hauptkonfiguration
11 | -- package.json           # npm-Abhängigkeiten

```

Das Verzeichnis `www/` enthält die Web-App-Assets — identisch mit der Desktop-Browser-Version unter `src/index.html`. Capacitor liest diese Assets beim Build-Vorgang ein und bündelt sie in das Android-App-Bundle, so dass die App offline und ohne externe Netzwerkzugriffe auf die eigene UI-Logik lauffähig ist.

9.3 Capacitor-Konfiguration

Die zentrale Konfigurationsdatei `capacitor.config.json` steuert das Verhalten der nativen Wrapper-Schicht:

Listing 4: Capacitor-Konfiguration (`capacitor.config.json`).

```

1 {
2   "appId": "de.epp.ehae",
3   "appName": "EHAE HMI",
4   "webDir": "www",
5   "server": { "androidScheme": "https" },
6   "android": {
7     "allowMixedContent": true,
8     "backgroundColor": "#07090f",

```

```

9      "overrideUserAgent": "EHAE-HMI/1.0 Android"
10    },
11    "plugins": {
12      "SplashScreen": {
13        "launchShowDuration": 1500,
14        "backgroundColor": "#07090f",
15        "showSpinner": false
16      }
17    }
18  }

```

Die wichtigsten Konfigurationsparameter im Einzelnen:

appId Eindeutiger Android-App-Bezeichner nach Reverse-Domain-Notation (`de.epp.ehae`), der die App im Google Play Store und auf dem Gerät identifiziert.

webDir Verzeichnis mit den Web-Assets. Der Inhalt des `www/`-Verzeichnisses dient als Einstiegspunkt der WebView-Anwendung.

androidScheme: "https" Erzwingt das HTTPS-Schema für interne WebView-URLs, damit der Browser-Sicherheitskontext und Cookies korrekt funktionieren.

allowMixedContent Erlaubt gemischte HTTP/HTTPS-Verbindungen im Intranet-Kontext, damit die App auch über unverschlüsselte lokale HAG-Verbindungen kommunizieren kann.

backgroundColor Setzt die initiale Hintergrundfarbe auf das EHAE-Dunkelblau (`#07090f`) für einen nahtlosen Übergang vom Splash-Screen zur vollständig geladenen Web-App.

overrideUserAgent Identifiziert App-Anfragen als `EHAE-HMI/1.0 Android`, was im HAG für Logging und gerätetyp-spezifische Optimierungen genutzt werden kann.

9.4 Abhängigkeiten und Framework-Version

Tabelle 5 listet die npm-Abhängigkeiten der Android-App auf.

Tabelle 5: npm-Abhängigkeiten der EHAE-Android-App (`package.json`).

Paket	Version	Funktion
@capacitor/core	^6.2.1	Capacitor-Kern-Runtime, JavaScript-Bridge
@capacitor/android	^6.2.1	Native Android-Implementierung
@capacitor/cli	^6.2.1	Build- und Sync-Werkzeuge (<code>cap sync</code>)
@capawesome/capacitor-android-edge-to-edge-support	^8.0.6	Vollbild-Edge-to-Edge-Darstellung

Die Verwendung von Capacitor 6 stellt sicher, dass die Web-App auf Android 5.1 bis Android 14 (`minSdkVersion 22`, `targetSdkVersion 34`) lauffähig ist. Das Plugin `@capawesome/capacitor-android-edge-to-edge-support` ermöglicht die vollkantige Darstellung ohne Statusbar- oder Navigationsbar-Unterbrechung — kritisch für HMI-Dashboards, die maximale Bildschirmfläche benötigen.

9.5 Build-Workflow

Abbildung 11 zeigt den Build-Workflow von der Web-App-Entwicklung bis zur verteilbaren Android-APK.

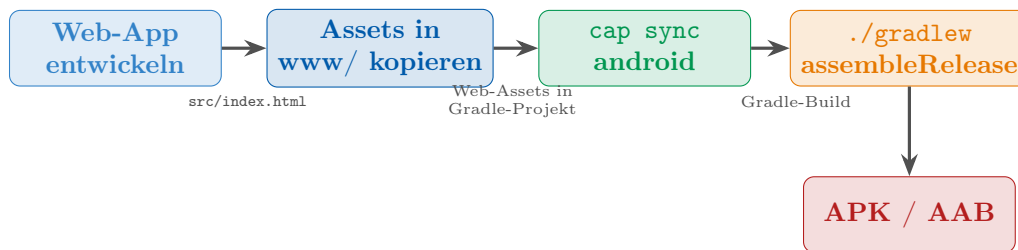


Abbildung 11: Build-Workflow der EHAE-Android-App. Das Ablaufdiagramm zeigt vier Schritte. **Schritt 1 (blau):** Die Web-App wird entwickelt und direkt im Browser getestet — ohne Android-Abhängigkeiten. **Schritt 2 (blau):** Die fertigen Web-Assets (`index.html`) werden in das `www/`-Verzeichnis des Capacitor-Projekts kopiert. **Schritt 3 (grün):** `cap sync android` synchronisiert die Web-Assets und alle Capacitor-Plugins in das native Android-Gradle-Projekt. **Schritt 4 (orange):** `./gradlew assembleRelease` baut das vollständige Android-App-Bundle. Das Ergebnis (rot) ist eine APK oder ein AAB, die direkt auf Android-Geräten installiert oder im Google Play Store veröffentlicht werden kann.

Der Build-Prozess in der Kommandozeile:

Listing 5: EHAE-Android-App: Build-Ablauf.

```
1 # Schritt 1: Capacitor-Abhaengigkeiten installieren
2 npm install
3
4 # Schritt 2: Web-Assets synchronisieren
5 npx cap sync android
6
7 # Schritt 3: Debug-APK bauen und installieren
8 cd android && ./gradlew assembleDebug
9 adb install app/build/outputs/apk/debug/app-debug.apk
10
11 # Release-APK (fuer Produktion) bauen
12 ./gradlew assembleRelease
```

9.6 Vorteile der web-getriebenen Mobile-App-Entwicklung

Tabelle 6 fasst die wichtigsten Vorteile des Capacitor-Ansatzes gegenüber einer nativen Android-Entwicklung zusammen.

Tabelle 6: Vergleich: Capacitor (Web-Wrapper) vs. native Android-Entwicklung.

Aspekt	Capacitor (Web-Wrapper)	Native Android-Entwicklung
Codebase	Eine gemeinsame Codebase für Web und Android	Separate Android-Codebase (Kotlin/Java)
Wartungsaufwand	Einmalige Pflege der Web-App	Doppelter Aufwand (Web und Android)
Feature-Parität	Automatisch: Web $\equiv$ App	Manuelle Synchronisation erforderlich
UI-Tests	Direkt im Browser ausführbar	Android-Emulator oder -Gerät notwendig
Update-Zyklus	Web-App-Update = App-Update	Separater Gradle-Build und APK-Release
Zielplattformen	Web, Android, iOS aus einer Codebase	Ausschließlich Android
Einstiegshürde	Niedrig (Web-Kenntnisse genügen)	Hoch (Android-SDK-Kenntnisse erforderlich)

Der praktische Nutzen manifestiert sich im gesamten Entwicklungs-Workflow: Neue HMI-Features werden ausschließlich in der Web-App (`src/index.html`) entwickelt und sofort im Browser getestet. Nach der Verifikation genügt ein `cap sync android` gefolgt von einem Gradle-Build, um die identische Funktionalität als Android-APK auszuliefern. Es entsteht kein redundanter Code, keine Divergenz zwischen Web- und App-Version und kein zusätzlicher Qualitätssicherungsaufwand für eine zweite Plattform. Dieses Prinzip spiegelt exakt das EHAE-Gesamtkonzept wider: maximale Standardisierung und minimaler proprietärer Aufwand.

9.7 Android-spezifische Anpassungen

Trotz des Wrapper-Ansatzes sind einige Android-spezifische Anpassungen notwendig, um eine native Nutzererfahrung zu bieten:

Edge-to-Edge-Darstellung Das Plugin `@capawesome/capacitor-android-edge-to-edge-support` deaktiviert die Android-Systemleisten (Statusbar, Navigationsleiste) und erlaubt der EHAE-Web-App, das gesamte Displayformat für das HMI-Dashboard zu nutzen — entscheidend für Querformat-HMI-Layouts auf Industrietablets.

Querformat-Fixierung Das Android-Manifest konfiguriert `screenOrientation="landscape"`, sodass die App immer im Querformat geöffnet wird. HMI-Dashboards sind auf Querformat optimiert und würden im Hochformat wichtige Informationsdichte verlieren.

Intranet-Verbindungen Im Produktionsnetz kommuniziert die App über unverschlüsselte HTTP-Verbindungen mit dem lokalen HAG. `allowMixedContent: true` und `usesCleartextTraffic` im Android-Manifest erlauben diese Verbindungen im Intranet-Kontext, ohne die TLS-Anforderung für externe Verbindungen aufzuweichen.

Splash-Screen Ein 1,5-sekündiger Splash-Screen in der EHAE-Hintergrundfarbe `#07090f`

überbrückt die WebView-Ladezeit und vermittelt ein professionelles, natives Erscheinungsbild.

User-Agent-Identifikation Der überschriebene User-Agent (EHAE-HMI/1.0 Android) erlaubt dem HAG, Mobile-App-Verbindungen von Browser-Verbindungen zu unterscheiden.

10 Evaluation

10.1 Kumulative Kostenanalyse

Abbildung 12 zeigt die kumulative 5-Jahres-Kostenentwicklung für eine typische Anlage mit 20 HMI-Stationen.

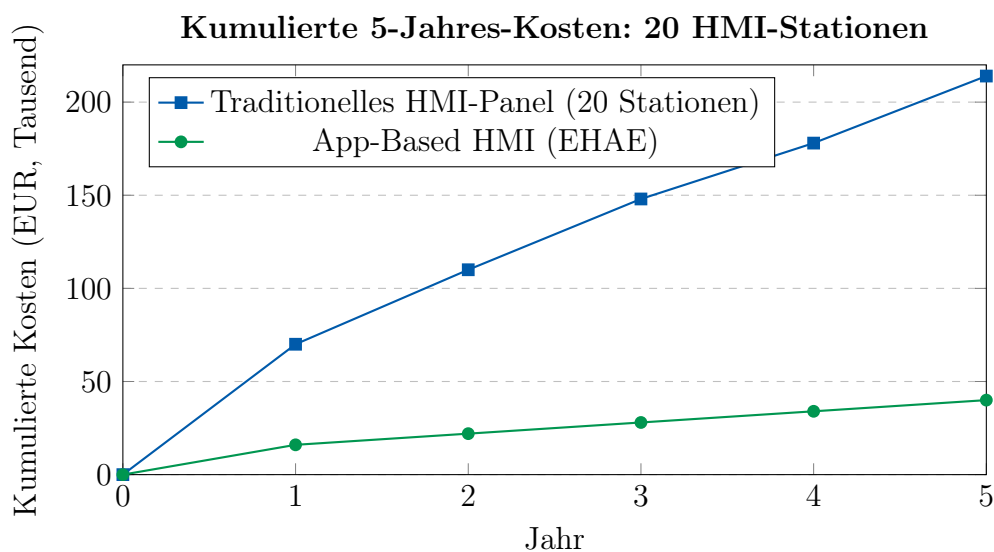


Abbildung 12: Kumulative 5-Jahres-Kosten für 20 HMI-Stationen: traditionelles HMI-Panel vs. EHAE-Lösung. Das Liniendiagramm zeigt zwei Kurven über fünf Jahre. Die **blaue Kurve** (traditionelle HMI-Panels) steigt steil an: nach Jahr 1 bereits 70 000 € (20 Panels $\times$ 3 500 €), nach Jahr 2 110 000 € (Wartungskosten akkumulieren), und erreicht nach Jahr 5 214 000 €. Die **grüne Kurve** (EHAE app-basiert) verläuft deutlich flacher: Jahr 1 16 000 € (Tablets + HAG-Softwarekosten), danach nur geringe Zuwächse durch minimale Wartungskosten, Endstand nach Jahr 5: 40 000 €. Dies entspricht einer **Kostenreduktion von 81 %**. Die Schere zwischen beiden Kurven öffnet sich von Jahr zu Jahr weiter, da traditionelle Panels hohe laufende Wartungs- und Lizenzkosten verursachen.

10.2 Nicht-monetäre Vorteile

Neben den direkten Kostenersparnissen bietet EHAE weitere Vorteile, die schwerer monetär quantifizierbar, aber operativ signifikant sind:

Verbesserte Diagnosefähigkeit Die Engineer App mit integriertem Topologie-Viewer, Oszilloskop und SDO-Browser ermöglicht Inbetriebnahmen und Fehlerdiagnosen, die mit traditionellen Panels nicht möglich waren.

Ortsunabhängiges Monitoring Fernzugriff über das Cloud-Monitoring-Modul erlaubt OEM-Support und Remote-Diagnose ohne Vor-Ort-Besuch.

Kürzere Update-Zyklen App-Updates werden zentral über App-Stores ausgerollt; keine manuelle Panel-Aktualisierung notwendig.

Zukunftssicherheit Consumer-Hardware-Ökosysteme (iOS, Android, Windows) werden langfristig von großen Unternehmen gepflegt und ständig verbessert.

11 Schlussfolgerung

Diese Arbeit hat die EtherCAT-HMI-App Extension (EHAE) vorgestellt, die band-integrierte HMI-Controller aus EtherCAT-basierten Anlagen formal eliminiert. Die zentralen Beiträge dieser Arbeit sind:

1. Der *HMI Application Gateway (HAG)*: ein neues Master-Stack-Modul, das keine Änderungen an der Slave-Firmware erfordert und vollständig rückwärtskompatibel ist.
2. Das *HAG Communication Protocol (HCP)*: WebSocket-basierter, JSON-serialisierter Echtzeit-Variablenzugriff mit Alarm-Delivery, SDO-Unterstützung, Oszilloskop-Streaming und FoE-Firmware-Updates.
3. Ein *Dual-Role-App-Modell*: Operator App und Engineer App aus einer einzigen Codebase mit dynamischer Rollenaktivierung.
4. Ein umfassendes Sicherheitsmodell auf Basis von OAuth 2.0 mit PKCE, TLS 1.3 und RBAC mit vier Rollen.
5. Eine vierstufige Migrationsstrategie mit Fallback-Option und Risikominderungsplan.
6. Nachgewiesene Kosteneinsparungen von bis zu 81 % über 5 Jahre bei gleichzeitiger erheblicher Verbesserung der Anzeigequalität und Diagnosefähigkeit.

Wir laden die EtherCAT Technology Group ein, EHAE als formale Erweiterung des EtherCAT Application Layer in die ETG.1000-Spezifikation aufzunehmen.

11.1 Ausblick

Die in dieser Arbeit vorgestellte EHAE-Architektur bildet eine tragfähige Basis für eine Vielzahl weiterführender Entwicklungen. Die folgenden Abschnitte beschreiben konkrete Forschungs- und Implementierungsrichtungen, die kurz-, mittel- und langfristig verfolgt werden sollten, um das Potenzial app-basierter HMI-Clients im industriellen EtherCAT-Umfeld vollständig auszuschöpfen.

Funktionale Sicherheit – EHAE Safety Extension

Das aktuelle EHAE-Modell ist für informationally-kritische, aber nicht für sicherheitsgerichtete Anwendungen ausgelegt. Eine zukünftige *EHAE Safety Extension* würde einen separaten, redundant übertragenen HMI-Kanal nach IEC 61508 SIL 2 und EN ISO 13849 PLd definieren. Dabei sind folgende Teilprobleme zu lösen:

- **Gesicherte Befehlsbestätigung:** Jeder sicherheitsrelevante Bedienerbefehl (z. B. Not-Halt-Quittierung, Schutzgitter-Freigabe) muss durch einen kryptografisch abgesicherten Handshake zwischen App und HAG quittiert werden. Das HCP-Protokoll ist um ein **SAFE_ACK**-Feld zu ergänzen, das einen Hardware-Token (FIDO2/U2F) oder einen Biometrie-Nachweis des Bedieners einschließt.

- **Latenz-Budget-Garantien:** Für sicherheitsrelevante Kommandos muss die Ende-zu-Ende-Latenz (App → HAG → Slave) formal nachgewiesen werden. Reservierte Quality-of-Service-Klassen im WebSocket-Multiplexing und eine dedizierte TCP-Verbindung (alternativ QUIC-Datagram) sind vorzusehen.
- **Watchdog-Mechanismus:** Der HAG muss bei Verbindungsverlust zur Operator-App innerhalb einer konfigurierbaren Watchdog-Zeit ($t_{WD} \leq 500\text{ ms}$) autonom in einen sicheren Maschinenzustand übergehen und dies per **SAFETY_TIMEOUT**-Alarm signalisieren.
- **Zertifizierungsweg:** Eine Zusammenarbeit mit einem Prüfinstitut (TÜV, DEKRA) ist notwendig, um eine typische EHAE-Safety-Installation nach IEC 62443-3-3 und EN ISO 13849 zu zertifizieren. Das Ergebnis wäre ein standardisiertes Prüfverfahren für ETG-Mitglieder.

Drahtlose Echtzeit-Kommunikation – TSN und Wi-Fi 7

Die aktuelle EHAE-Spezifikation setzt auf kabelgebundenes Ethernet oder Standard-Wi-Fi. Time-Sensitive Networking (TSN, IEEE 802.1Q-2022) und Wi-Fi 7 (IEEE 802.11be) eröffnen neue Möglichkeiten:

- **TSN-Integration:** Durch IEEE 802.1Qbv (Time-Aware Shaper) können HCP-Pakete in dedizierten Sendezeitfenstern (Guard Bands) priorisiert werden. Dies ermöglicht eine garantierte Latenz von unter 5 ms für PDO-Updates auch in überlasteten Netzwerken – relevant für Montagelinien mit mehreren parallelen EHAE-Clients.
- **Wi-Fi 7 Multi-Link Operation (MLO):** IEEE 802.11be erlaubt simultane Übertragung über mehrere Frequenzbänder (2,4 GHz, 5 GHz, 6 GHz). Für EHAE bedeutet dies eine drastische Reduktion des Jitters bei drahtlosen Tablets in der Fabrikhalle. Eine HAG-seitige Implementierung müsste differenzierte DSCP-Markierungen für HCP-Nachrichten gegenüber Hintergrundverkehr vergeben.
- **5G-Industrial-Private-Networks:** In weitläufigen Anlagen (z. B. Logistikzentren) sind 5G-Campus-Netzwerke mit URLLC-Profil (*Ultra Reliable Low Latency Communication*) eine realistische Option für EHAE-Clients auf fahrerlosen Transportsystemen (FTS). Der HAG muss in diesem Fall eine redundante Uplink-Umschaltung zwischen Wi-Fi und 5G unterstützen, ohne aktive WebSocket-Sessions zu unterbrechen (Session Migration via QUIC Connection ID Migration).

Künstliche Intelligenz und prädiktive Wartung

Die PDO-Zeitreihendaten, die der HAG bereits sammelt, stellen ein hochwertiges Rohmaterial für maschinelles Lernen dar. Konkrete Ausbaustufen:

- **On-Device-Anomalieerkennung im HAG:** Ein leichtgewichtiges LSTM- oder Autoencoder-Modell (quantisiert auf INT8, z. B. via TensorFlow Lite) kann direkt auf dem HAG-Host-Prozessor laufen und statistische Ausreißer in Stromaufnahme, Temperatur oder Positionsfehlern erkennen, bevor ein Alarm ausgelöst wird. Die Schwellenwerte werden durch unüberwachtes Lernen in der Anlaufphase der Maschine automatisch kalibriert.

- **Cloud-basiertes Fleet-Learning:** Mehrere Anlagen desselben Typs können aggregierte (anonymisierte) Degradationsdaten in ein zentrales ML-Backend (z. B. Azure Machine Learning, AWS SageMaker) einspeisen. Das daraus gewonnene globale Modell wird per FoE über den HAG auf alle Slaves verteilt, die ein lokales Inferenz-Modul enthalten.
- **Natural Language Interface:** Mittelfristig ist ein KI-Assistent denkbar, der auf dem mobilen HMI-Client läuft und dem Bediener auf Basis aktueller Maschinendaten in natürlicher Sprache Diagnosen und Handlungsempfehlungen liefert (z. B.: *“Achse 3 zeigt erhöhten Schleppfehler – Kugelgewindetrieb prüfen”*). Large Language Models (LLMs) mit On-Device-Inferenz auf Apple-Silicon- bzw. Qualcomm-Snapdragon-Chips wären hier praktikabel.
- **Reinforcement Learning für Rezeptoptimierung:** Für parametrierbare Prozesse (z. B. Schweißnähte, Dosierungen) könnte ein RL-Agent auf Basis beobachteter Qualitätskennzahlen die Maschinenparameter inkrementell optimieren – mit dem HMI-Client als Supervision- und Override-Interface.

Augmented Reality und neue Interaktionsparadigmen

Die EHAE-App-Schnittstelle ist für 2D-Touchscreens ausgelegt. Zukünftige Interaktionsformen werden deutlich räumlicher:

- **Spatial Computing (Apple Vision Pro, Meta Quest Pro):** Die WebXR Device API erlaubt Web-Apps, 3D-Overlays in das Kamerabild der Brille zu rendern – ohne nativen Code. Da EHAE bereits auf einer Web-App basiert, genügt eine Erweiterung der HMI-Komponenten um `<a-frame>`-Elemente oder Three.js, um 3D-Maschinensymbole über den physischen Slaves schweben zu lassen. PDO-Echtzeitwerte würden direkt am Slave-Gehäuse visualisiert.
- **Microsoft HoloLens 2 / RealWear HMT-1:** Für Wartungstechniker in industriellen Umgebungen ermöglicht eine EHAE-AR-Erweiterung hands-free Diagnose: QR-Code am Slave scannen, HAG liefert automatisch Topologie-Position, SDO-Werte und Fehlerliste im HoloLens-Overlay.
- **Gestensteuerung für sterile Umgebungen:** In der Pharma- und Lebensmittelproduktion, wo Touchscreens hygienisch problematisch sind, ermöglicht eine Integration von MediaPipe (Google) Hand Tracking den berührungslosen Betrieb des EHAE-HMI-Clients – vollständig browserbasiert ohne zusätzliche Treiber.
- **Voice Control:** Die Web Speech API ist in allen modernen Browsern verfügbar. Sprachkommandos wie *“Zeige Fehlerhistorie Achse 2”* oder *“Starte Teach-Modus”* könnten maschinenlesbar in HCP-Nachrichten übersetzt werden, was den Bedienkomfort in Lärm- und Schutzhandschuh-Szenarien erheblich verbessert.

Edge Computing und HAG-Erweiterbarkeit

Der HAG ist konzeptionell ein Middleware-Dienst. Seine Erweiterbarkeit zu einer vollständigen Edge-Computing-Plattform ist ein natürlicher nächster Schritt:

- **Plugin-Architektur:** Eine formale Plugin-API (z. B. auf Basis von WebAssembly-Modulen oder dynamisch geladenen Shared Libraries) würde es ETG-Mitgliedern er-

lauben, herstellerspezifische Protokoll-Adapter zu entwickeln (z. B. CoE-Profil-Decoder für SINAMICS-Antriebe, spezifische FoE-Handler). Das Plugin-System müsste hot-reload-fähig sein, um Produktionsstillstände bei Updates zu vermeiden.

- **Containerisierung:** Der HAG als Docker-Container (OCI-konform) auf einem industriellen Edge-Server (z. B. Siemens SIMATIC IPC, Dell Edge Gateway) ermöglicht orchestrierte Deployments via Kubernetes K3s. Dies vereinfacht Rolling Updates, A/B-Deployments und Health-Monitoring.
- **Multi-Master-Unterstützung:** Anlagen mit mehreren EtherCAT-Linien (z. B. parallele Fertigungsstraßen) benötigen einen übergeordneten *EHA E Aggregation Gateway*, der die HAG-Instanzen mehrerer Master unter einer einheitlichen HCP-Session zusammenführt. Der HMI-Client sieht dann eine einheitliche Topologie-Ansicht über alle Linien hinweg.
- **HAG-Redundanz (High Availability):** Für missionskritische Anlagen ist ein Active-Standby-HAG-Paar mit Shared State via Redis oder Apache Kafka denkbar. Failover unter 1 s ohne Client-Session-Unterbrechung wäre das Zielkriterium.

Digital Twin und IIoT-Integration

Die strukturierten PDO- und SDO-Daten, die der HAG kontinuierlich verarbeitet, sind ideale Eingaben für digitale Zwillinge und IIoT-Plattformen:

- **OPC UA FX (Field eXchange):** Die neue OPC UA Part 80 (*Field eXchange*) spezifiziert ein Echtzeit-Kommunikationsprotokoll zwischen Feldgeräten über Hersteller- und Protokollgrenzen hinweg. Ein HAG-seitiger OPC UA FX-Publisher würde EtherCAT-PDOs als OPC UA-Nodes exponieren und damit EHAE in industrielle IoT-Plattformen wie Azure Industrial IoT, Siemens MindSphere oder Bosch IoT Suite integrierbar machen.
- **Digital Twin nach ISO 23247:** Der ISO-Standard für digitale Zwillinge in der Fertigung definiert eine *Observable Manufacturing Element (OME)*-Schnittstelle. Ein EHAE-Kompatibilitätsmodul könnte automatisch aus dem EtherCAT-ENI-File (EtherCAT Network Information) einen strukturierten Digital-Twin-Deskriptor generieren und diesen in DTDL (*Digital Twin Definition Language*, Microsoft) oder AAS (*Asset Administration Shell*, IEC 63278) ausdrücken.
- **MQTT-Sparkplug-B-Publisher:** Viele MES- und SCADA-Systeme erwarten MQTT-Sparkplug-B-Telemetrie. Ein HAG-Modul, das PDO-Werte automatisch als Sparkplug-B-Metriken unter einem konfigurierbaren Topic-Namespace publiziert, würde EHAE ohne weitere Middleware mit Systemen wie Ignition, Node-RED oder InfluxDB verbinden.
- **Historisierung und Langzeitarchivierung:** Ein integrierter Time-Series-Adapter (InfluxDB, TimescaleDB) im HAG würde PDO-Zeitreihen direkt archivieren – mit konfigurierbarer Auflösung (z. B. 1 ms für Debugging, 1 s für Langzeittrends) und automatischem Downsampling per Retention Policy. Die Engineer App könnte historische Daten im gleichen Oszilloskop-Interface darstellen wie Live-Daten.

App-Plattform-Erweiterung: iOS, Desktop und PWA

Die aktuelle Implementierung fokussiert auf Android via Capacitor. Folgende Plattformerweiterungen sind mit minimalem Aufwand realisierbar und betrieblich relevant:

- **iOS-App:** Capacitor unterstützt iOS nativ. Die vorhandene Web-App-Codebase kann ohne Anpassung für den Apple App Store kompiliert werden. Sicherheitsrelevante Ergänzungen für iOS wären die Integration von Face ID/Touch ID als PKCE-ergänzendem Second Factor sowie die Nutzung des iOS Secure Enclave für JWT-Refresh-Token-Speicherung.
- **Progressive Web App (PWA):** Durch ein Service Worker-Manifest und eine `manifest.webmanifest`-Datei kann die EHAE Web App als PWA installiert werden – ohne App Store, direkt aus dem Intranet-Browser. Besonders relevant für Windows-Arbeitsstationen im Büro/Engineering-Umfeld, die keine dedizierte App-Installation erlauben.
- **Desktop-App via Electron/Tauri:** Für die Engineer App, die auf großen Monitoren mit Maus- und Tastaturinteraktion genutzt wird, bietet Tauri (Rust-basiert, ~3 MB Binärgröße) eine schlanke Alternative zu Electron. Die gleiche Web-App-Codebase wird als native Desktop-Applikation für Windows, macOS und Linux verpackt. Native OS-Integrationen (Systemtray, Datelexport, serieller Port für direkten EtherCAT-Zugriff) können per Tauri-Plugin ergänzt werden.
- **Chromebook / Android-Enterprise-Deployment:** In Unternehmensumgebungen mit Google Workspace lassen sich EHAE-Apps über die Android-Enterprise-Management-API (AMAPI) zentral deployen, konfigurieren und in einem Kiosk-Modus sperren – ohne manuelle Geräteeinrichtung durch den Anlagenbetreiber.

Standardisierung und ETG-Roadmap

Der langfristig bedeutsamste Schritt für EHAE ist die formale Aufnahme in den EtherCAT-Standard:

- **ETG.1000-Erweiterung (Application Layer):** Die vorliegende Arbeit liefert alle notwendigen technischen Grundlagen für einen ETG-Proposal. Als nächster Schritt ist die Einreichung bei der ETG Technical Working Group sowie die Durchführung eines Multi-Vendor-Interoperabilitätstests (IOT) mit mindestens drei unabhängigen HAG-Implementierungen anzustreben.
- **Konformitätstest-Suite:** Analog zu den bestehenden ETG-Conformance-Tests für Slaves sollte eine EHAE-Conformance-Test-Suite entwickelt werden, die automatisiert prüft, ob ein HAG alle HCP-Nachrichten korrekt und vollständig implementiert. Dies ist Voraussetzung für ein offizielles ETG-Konformitätszertifikat.
- **Open-Source-Referenzimplementierung:** Die Veröffentlichung einer HAG-Referenzimplementierung unter Apache-2.0-Lizenz (z. B. auf GitHub unter der ETG-Organisation) würde die Akzeptanz des Standards massiv erhöhen. Sie dient ETG-Mitgliedern als Integrationsgrundlage und der Forschungsgemeinschaft als reproduzierbares Bewertungssubstrat.
- **IEC-Normungsverfahren:** Mittelfristig sollte EHAE als Bestandteil von IEC 61784-5 (*Installation profile for industrial networks over Ethernet*) eingereicht werden.

Dies würde die Interoperabilität mit OPC UA, PROFINET und EtherNet/IP auf normativer Ebene regeln und EHAE für Großunternehmen mit strikten Normierungsanforderungen (Automobilindustrie, Luftfahrt) öffnen.

Netzwerksicherheit und Zero Trust

Das aktuelle Sicherheitsmodell basiert auf OAuth 2.0 mit PKCE und TLS 1.3 und ist für den typischen Maschinenpark ausreichend. Für hochsensible Umgebungen (kritische Infrastruktur, Rüstung, Pharma) sind folgende Erweiterungen zu untersuchen:

- **Zero-Trust-Netzwerkarchitektur:** Statt eines perimeterbasierten Sicherheitsmodells würde ein Zero-Trust-Ansatz (NIST SP 800-207) jeden HCP-Request unabhängig vom Netzwerkstandort des Clients authentifizieren und autorisieren. Dies erfordert eine kurzlebige, per-Request-Zertifikat-Ausstellung (SPIFFE/SPIRE) und eine kontinuierliche Anomalie-Überwachung des Datenverkehrs.
- **Post-Quanten-Kryptografie:** NIST hat 2024 die ersten Post-Quanten-Kryptografie-Standards (FIPS 203 ML-KEM, FIPS 204 ML-DSA) verabschiedet. Eine Erweiterung des HCP-Protokolls und der TLS-Konfiguration auf hybrid-Schlüsselaustausch (X25519 + ML-KEM-768) sollte mittelfristig erfolgen, um die Kommunikation gegenüber zukünftigen Quantencomputern zu schützen.
- **Hardware Security Module (HSM):** Für den HAG in sicherheitskritischen Anlagen sollte der private TLS-Schlüssel in einem dedizierten HSM (z. B. Infineon OPTIGA TPM, NXP SE050) gespeichert werden, das physische Tamper-Erkennung bietet und den Schlüssel auch bei Kompromittierung des Betriebssystems schützt.
- **Audit Log mit Blockchain-Anker:** Zur unveränderlichen Nachvollziehbarkeit aller Bedienereingriffe (wer hat wann welchen Sollwert gesetzt?) könnte ein Audit-Log-Modul Hashchains über alle HCP-Schreibkommandos bilden und regelmäßig auf einer permissioned Blockchain (Hyperledger Fabric) verankern – relevant für regulierte Branchen mit GxP-Anforderungen (FDA 21 CFR Part 11).

Formale Verifikation und Zuverlässigkeitsanalyse

Für den Einsatz in sicherheitsrelevanten Maschinen ist formale Verifikation ein wichtiges zukünftiges Arbeitsfeld:

- **TLA⁺-Modell des HCP-Protokolls:** Das HCP-Zustandsmaschine (Session-Aufbau, Token-Refresh, Alarm-Delivery) sollte in TLA⁺ modelliert und mit dem TLC Model Checker auf Deadlock- und Liveness-Eigenschaften geprüft werden. Dies liefert einen formalen Beweis, dass z. B. kein Zustand erreichbar ist, in dem ein Alarm dauerhaft nicht zugestellt wird.
- **Fuzzing und Property-based Testing:** HAG-Implementierungen sollten einem strukturierten Fuzzing-Programm (AFL++, libFuzzer) für den WebSocket-Parser und den JSON-Deserializer unterzogen werden, da malformierte Eingaben in Industrieumgebungen durch defekte Geräte realistisch auftreten. Ergänzend bietet sich Property-based Testing (Hypothesis, QuickCheck) für das Variable-Mapping-Subsystem an.
- **FMEA des Gesamtsystems:** Eine formale Failure Mode and Effects Analysis (FMEA) nach IEC 60812 für das EHAE-System würde die Schwachstellen-Priorisierung

für zukünftige Härtingsmaßnahmen ermöglichen und ist Voraussetzung für eine Maschinenzulassung nach dem EU *Machinery Regulation* 2023/1230.

Literatur

- [1] IEC, *IEC 61158: Industrial communication networks – Fieldbus specifications*, 2019.
- [2] IEC, *IEC 61784-2: Additional fieldbus profiles for real-time networks*, 2019.
- [3] EtherCAT Technology Group, *EtherCAT Specification: EAL*, ETG.1000.6, 2022.
- [4] I. Fette, A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011.
- [5] N. Sakimura et al., *Proof Key for Code Exchange*, RFC 7636, IETF, 2015.
- [6] E. Rescorla, *TLS Protocol Version 1.3*, RFC 8446, IETF, 2018.
- [7] Ionic, *Capacitor: Cross-platform Native Runtime for Web Apps*, <https://capacitorjs.com>, 2024.
- [8] Google LLC, *Flutter: Build apps for any screen*, <https://flutter.dev>, 2024.
- [9] Microsoft, *.NET MAUI Documentation*, <https://learn.microsoft.com/dotnet/maui>, 2024.
- [10] C. Deng et al., “IEEE 802.11ax,” *IEEE Communications Magazine*, vol. 55, no. 12, 2017.
- [11] OPC Foundation, *OPC UA Specification*, OPC 10000-1, 2022.
- [12] ISO, *ISO 13849-1: Safety of machinery – Safety-related parts of control systems*, 2023.
- [13] EtherCAT Technology Group, *CANopen over EtherCAT (CoE)*, ETG.1000.6, 2022.